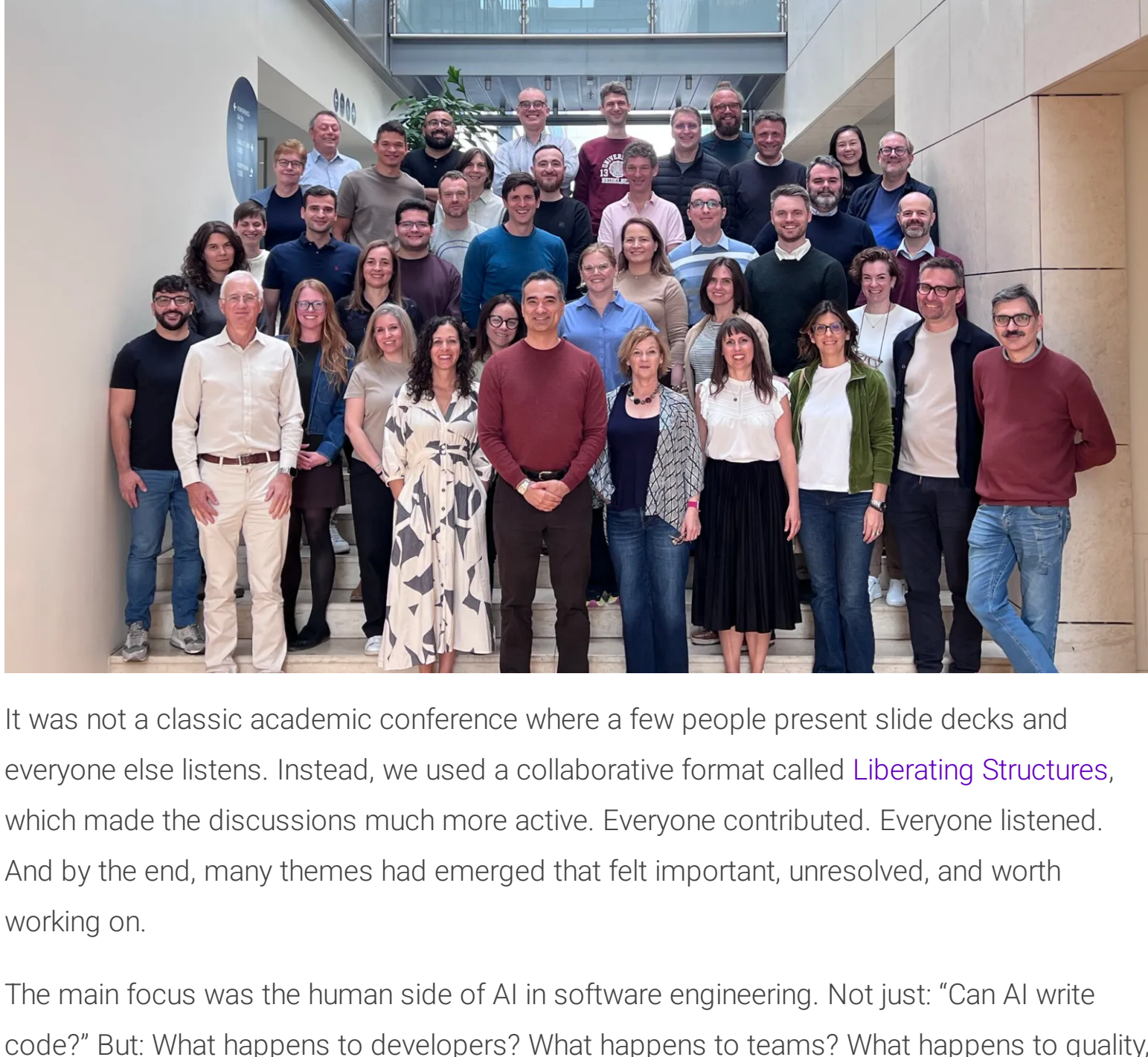


What 40 Software Engineering Researchers Talk About When They Talk About AI

In May, I joined about 40 leading software engineering researchers from academia and industry at Aalborg University in Copenhagen for the 3rd Copenhagen Symposium on Human-Centered AI Adoption in Software Engineering, organized by [Prof. Daniel Russo](#).

For three days, we talked about [how AI is actually changing software engineering](#), and what is relevant to understand, explore, and study. For me, it was also a time to reconnect with previous colleagues and meet many new inspiring people.



It was not a classic academic conference where a few people present slide decks and everyone else listens. Instead, we used a collaborative format called [Liberating Structures](#), which made the discussions much more active. Everyone contributed. Everyone listened. And by the end, many themes had emerged that felt important, unresolved, and worth working on.

The main focus was the human side of AI in software engineering. Not just: "Can AI write code?" But: What happens to developers? What happens to teams? What happens to quality, understanding, business value, learning, and responsibility?

Everyone was excited, because these times are unprecedented. We have more questions to explore, and more insights to reveal, than ever before.

Here are the themes we talked about that I'm most excited about.

Developer work and careers are shifting, but not everyone is excited about it

One big theme was developer identity, developer work, and the future of software engineering.

While some proclaim that [coding is dead](#), others see the developer role evolving. But for many developers, coding is not just a task. It is a craft and a passion. It is where they think, explore, learn, and create.

And while research suggests that developers only spend a part of their time writing and reviewing code, it is not enough to say: "Well, if they lose that part, it is not a big deal."

Developers consistently report wanting to spend more time on problem solving, learning, and making things. And while AI promises to take over the "boring" parts, it often also takes over some of the meaningful and creative parts, leaving developers with even more of the work they did not want to do in the first place.

[Tom Zimmermann](#), professor at UC Irvine, whose research group is called the Capybara Science Lab — who wouldn't like to join that — was at the symposium. He recently published a study about [the future of the developer profession](#), where he also discusses how the role is changing and how developers are adapting to the new AI-native or AI-first paradigm.

So, if AI takes over more of the actual code writing, reviewing, and deployment, the developer role may shift toward specifying, integrating, prompting, validating, and making higher-level decisions.

For some people, that feels like a natural evolution.

For others, it feels like losing the part of the job they enjoyed most.

I also met [Annie Vella](#), who took on the question of the developer's new role and tasks as one of her key PhD research questions. She also wrote about this in [The Middle Loop](#). Annie is a distinguished engineer at Westpac, one of the biggest banks in New Zealand, and she told me how many of her colleagues and friends are now struggling and finding themselves in an identity crisis. She is also thinking deeply about which career paths are still left for people whose current work is increasingly replaced by AI, and what responsibilities employers should take on.

What skills will matter one year from now? What responsibility do employers have to help developers adapt?

How do we prepare students and junior developers for the work?

[Arie van Deursen](#), my PhD advisor and professor at Delft University of Technology, was also deeply concerned about the future of developers. He stressed that we have to understand the implications of these changes in tasks and responsibilities, and that it is important to show development and growth opportunities that are aligned with people's interests and capabilities.

And naturally, as a professor at a university, another question becomes extremely important: Which skills and competencies should universities teach? How can they prepare the next generation of developers to be successful?

Connected to that is the junior developer problem. If we remove too much of the work that keeps people close to code, how will developers learn their craft and how will they evolve into future senior engineers? It is clear that AI still needs quite a bit of babysitting. The engineers steering the AI engine need deep technical understanding to produce good output. But how will the next generation get that technical knowledge and experience?

Less fun, more responsibility: the verification tax

Another aspect of this shift is that developers may spend less time writing code themselves, but more time checking whether generated code is correct, secure, maintainable, and aligned with the existing system.

Instead of spending two hours writing code, a developer may spend two hours verifying code.

This is especially tricky because AI-generated code often looks plausible. It may be syntactically correct. It may even pass some tests. But it can still be subtly wrong.

It was interesting to hear different opinions and ideas about how work will shift, especially in the context of code reviews. Some participants strongly believed that the era of code reviews is over. Others argued that we may need to go back to more formal code-inspection-like techniques. Some believed code reviews are more important than ever. But everyone agreed that the amount of code generated by AI is becoming harder and harder to review meaningfully.

I very much enjoyed talking to [Sílvia Abrahão](#), Full Professor of Software Engineering at Universitat Politècnica de València, about software quality and code reviews in the era of AI.

Some questions she investigates in her research group are: How does AI improve or complicate code reviews? Are AI comments useful to humans? And what does review even mean when code is increasingly generated?

When we look at what industry research like the [DORA ROI of AI-assisted Software Development report](#) says, it aligns with the idea that new skills and practices need to evolve. DORA explicitly calls out that teams need time to learn new workflows, review generated code, and adapt downstream processes such as testing and change approval, and they call this the J-Curve of AI adoption. The report also calls out the verification tax: developers invest time reviewing generated code because they have to deal with hallucinations, trustworthiness concerns, and the increased volume of code being produced.

What system artefacts will be or stay important going forward

So now we may spend less time on the "easy" part, which is writing the code ourselves, and more time on the hard part: verifying and understanding code written — or generated — by someone else.

If we span this idea even further, we might ask whether looking at code will still be relevant in the future. Or will generated code become the new bytecode, and we will inspect other artifacts at a higher level, such as UML diagrams — hello and welcome back — specifications — oh no, who wants to do that — or automated test suites, instrumentation, monitoring, and observability? What parts of the system do we still have to know and understand very well, and which parts can we trust that the machines can take care of?

This brings me directly to another very interesting theme that was discussed a lot: cognitive debt.

Cognitive debt may become as important as technical debt

If developers do not write the code, and maybe either do not review the code, or are not able to meaningfully review it because the task becomes too complex and cumbersome, what will they still know about how the system really works?

What research has already shown is that when AI generates code faster than humans can understand it, the team can slowly lose its mental model of the system. The code might work today, but nobody knows why it works, where the risks are, or what assumptions are hidden inside it.

[Margaret-Anne Storey](#) has written exactly about this in the context of [cognitive debt and intent debt](#). The idea is that AI can create or amplify gaps between the code, the intent behind the code, and the team's understanding of both.

Also during the symposium, this was a hot topic. I'm excited to be working with many colleagues on this over the coming year, including [Bianca Trinkenreich](#), [Alexander Serebrenik](#), [Margaret-Anne Storey](#), [Sarah Inman](#), [Tom Zimmermann](#), and many others.

Future tools may become better at documenting intent, tracing decisions, and explaining system behavior. But we should not assume that understanding will come for free. In fact, as everything gets faster, and often looks great from the outside but messy from the inside, this may become more relevant than ever.

Misconceptions and the complexity of the AI landscape

The final theme I want to mention is about misconceptions and the complexity of the AI landscape.

Really understanding AI models is already challenging. But that is not all. The whole landscape has become incredibly complex.

There are many models out there, and while we try to measure and understand their capabilities using [benchmarks](#), we know that benchmarks are limited. Even if we only talk about frontier models, we already have several different vendors. If we add the tools and harnesses around them, it becomes exponentially more complex.

Are you using Sonnet or Opus? GPT-5, GPT-5.4, or GPT-Codex? Are you using GPT via Copilot, or via the Codex app? Are you a CLI user? Does that matter?

Yes, it does.

It is not only the pure output of the LLM model. It is also about what the harness and the environment enable, prevent, steer, and change.

Even [Anthropic themselves](#), [calls out that problem](#): "One of the most common misconceptions about Claude Code is that its capabilities are solely defined by the model used."

[Prompt engineering](#) was a big topic only a few months ago. But now, for many tools, it may matter less than we think, because your original prompt may never hit the model unchanged. The prompt is rewritten. How? You may not know.

Similarly, for some tools, you may not know which actual model executed your task, how large the context was, or how many files were considered. Claude may have looked at more than 20 files before, and now maybe only at five or seven. Not everything is transparent. In fact, more and more becomes opaque. Often on purpose.

Developers feel this. A setup that worked well yesterday may suddenly struggle today. Probably, the underlying technology changed. [Anthropic says](#): "As models evolve, instructions written for your current model can work against a future one."

So having a good understanding of what you are dealing with — the capabilities of your model, your tooling, and your environment — is not only important, it is crucial.

Understanding model capabilities can save you money

As tokens become more and more expensive, we have to understand the capabilities of models better, but also how we should best interact with each model. Should we use a frontier model for planning, and then let a less powerful model implement, as is often suggested? Does that actually lead to better outcomes? Or should we redefine our prompts? And if yes, in which way, for which model, and which harness?

This theme was extremely interesting to me, and I plan to spend more time investigating it. But there was no clear path forward from a research perspective, because it is hard to get good data. Existing data sources may not be enough to investigate this properly from an empirical standpoint. So, for now, there is no concrete path from this group to take work on [user misconceptions of LLMs](#) further, although this direction is very relevant.

There are so many questions, and the complexity of evaluating these systems is high. Right now, developers often rely on anecdotal knowledge. And once they finally get a better feeling for the tools and technologies, everything shifts again, and they have to start from square one.

So, where does this leave us?

There were, of course, many other discussions going on.

How much human involvement do we want to keep, and for which tasks? How should AI be used, or not used, for doing research or evaluating research? How should we influence the future? Do we need guidelines like the [Copenhagen Manifesto](#)? And if yes, who gets to define them?

I left Copenhagen with more questions than answers. Which is the best that can happen if you are a researcher by heart.

After all that I felt incredibly inspired and curious, with too many plans to follow up on than are humanly possible. So I am very thankful for the invitation and the opportunity, and I hope we can answer at least a few of those questions in the coming year, and that I can also leave a small dent in this very broad, fast-changing research landscape.

Updated on May 19th, 2026



Dr. Michaela Greiler makes code reviews a team's superpower through her [code review workshops](#). She has worked with teams from Microsoft, National Instruments, Metro Systems, Flutter, Wix and many more. [Learn more](#).

Join +3500 devs improving their code reviews

Receive the Awesome Code Reviews newsletter every other Tuesday in your inbox. It's packed with research based insights and tips.

I want in!

Email Address

First Name

Let's Go!

Search

Archives

- May 2026
- February 2026
- January 2025
- June 2024
- January 2024
- November 2023
- January 2022
- February 2021
- December 2020
- November 2020
- September 2020
- April 2020
- March 2020
- February 2020
- January 2020
- October 2019
- September 2019
- August 2019
- July 2019
- June 2019
- May 2019
- April 2019
- March 2019
- February 2019
- January 2019
- October 2016
- September 2016
- May 2016
- September 2014
- February 2013
- June 2012
- November 2010



